



02456 – Week 5

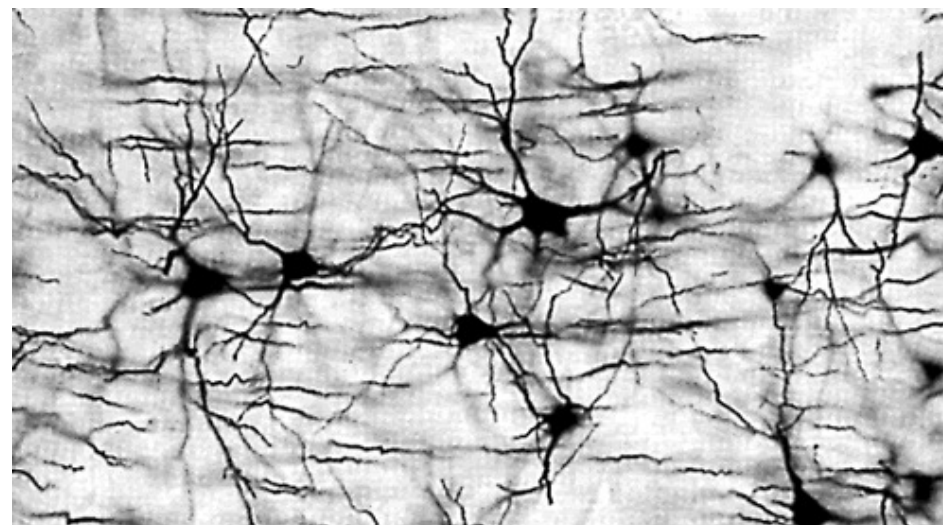
Recurrent

Neural Networks

Jes Frellsen

Technical University of Denmark

22 September 2025



Menu of the day

- Week 1: Neural nets
- Week 2: Learning
- Week 3: Tricks of The Trade
- Week 4: CNNs
- **Week 5: RNNs**
- Week 6: Transformers
- Week 7: Unsupervised
- Week 8: Mini-project

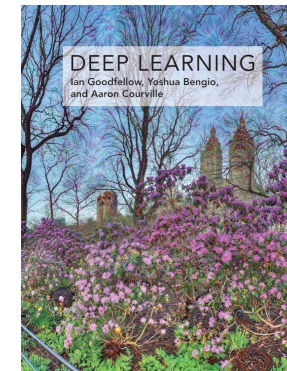
Architectures for
sequential data

Lecture

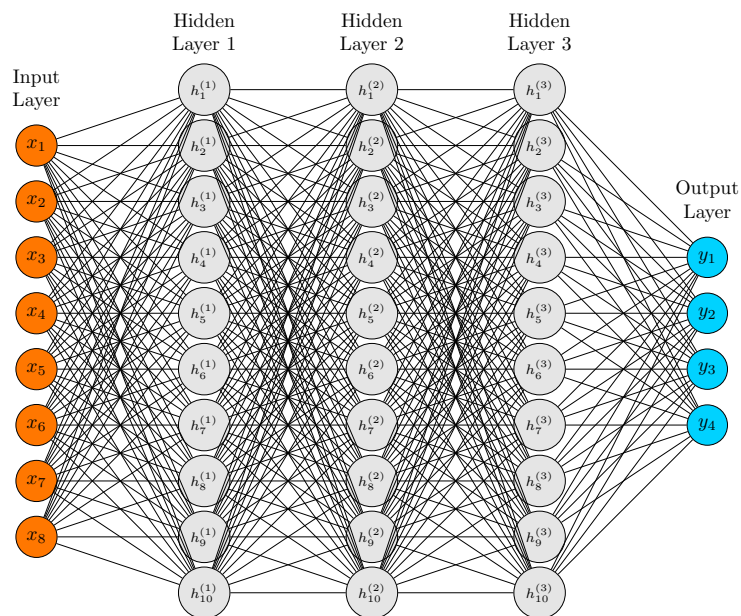
- Motivation for RNNs
- Details of RNNs
- Ch 10 in Goodfellow et al.

Exercises

- **Notebooks 5.1:**
 - RNN with Nanograd
 - LSTM with Nanograd & **PyTorch!**
- **Problems:** None



Recap: Neural networks and learning



We can write the **output of layer ℓ** as

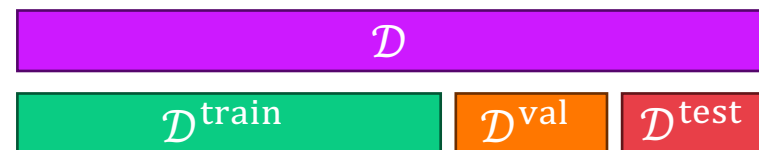
$$f^{(\ell)}(\mathbf{h}) = \sigma(W^{(\ell)}\mathbf{h} + \mathbf{b}^{(\ell)}),$$

where σ is a non-linear **activation function**.

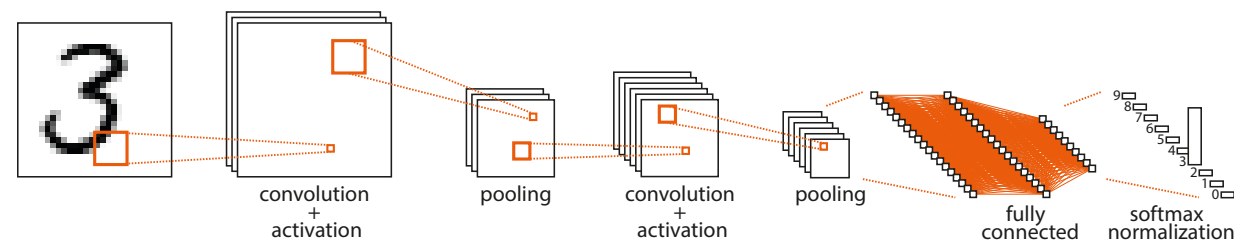
The **joint function** is then

$$f_{\phi}(\mathbf{x}) = \mathbf{y} = f^{(4)}\left(f^{(3)}\left(f^{(2)}\left(f^{(1)}(\mathbf{x})\right)\right)\right)$$

- A **loss function** $L(\phi)$ captures the **mismatch** of $f_{\phi}(\mathbf{x}_i)$ and $\mathbf{y}_i$: $\hat{\phi} = \operatorname{argmin}_{\phi} L(\phi)$
- Use the **NLL** $L(\phi) = -\sum_i \log p(\mathbf{y}_i | f_{\phi}(\mathbf{x}_i))$
- Use **gradient descent** to find $\hat{\phi}$
- Split **data set** in three
 - **Val**: Select hyper parameters
 - **Test**: estimate generalisation error



- Have special architectures for, e.g., images



Examples of sequence data

Speech recognition



^y
“The quick brown fox jumped
over the lazy dog.”

Music generation



Sentiment classification

“There is nothing to like
in this movie.”



DNA sequence analysis

AGCCCCTGTGAGGAACTAG



AG**CCCCTGTGAGGAACTAG**

Machine translation

Voulez-vous chanter avec
moi?



Do you want to sing with
me?

Video activity recognition



Running

Name entity recognition

Yesterday, Harry Potter
met Hermione Granger.

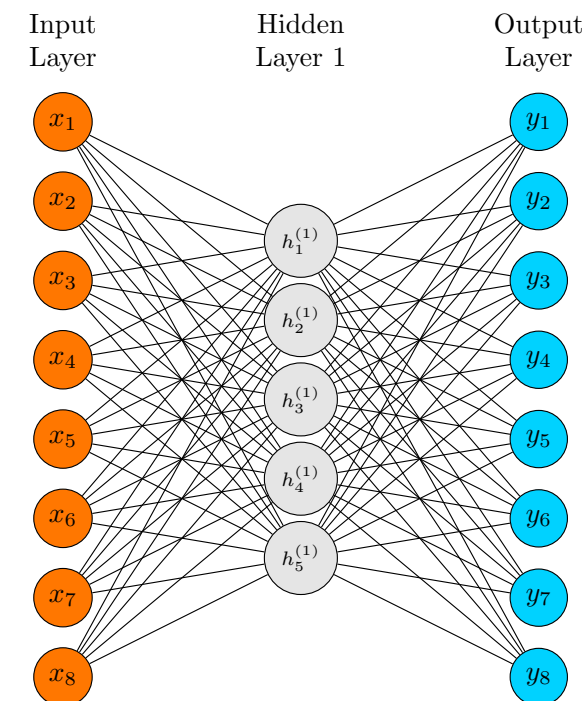


Yesterday, **Harry Potter**
met **Hermione Granger**.
Andrew Ng

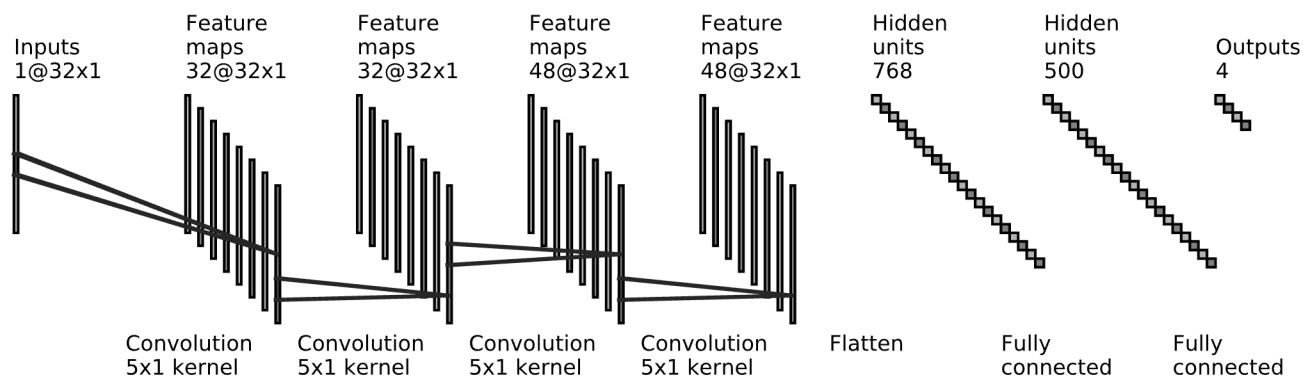
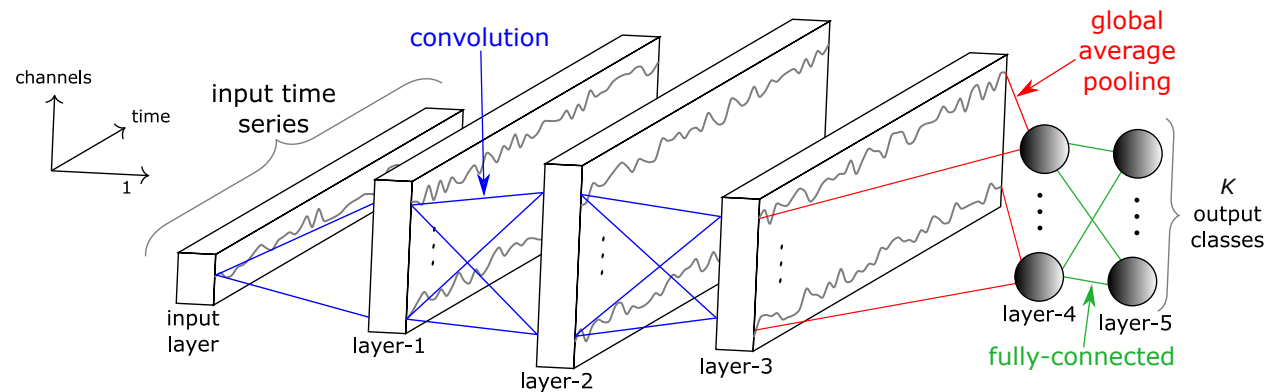
How to handle sequential data with neural networks?

- We want to do supervised learning on data pairs $(\mathbf{x}, \mathbf{y})$, where
 - The **input** $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_T)$ is a **sequence**, and $\mathbf{x}_t$ can be a vector
 - The **output** $\mathbf{y} = (\mathbf{y}_1, \dots, \mathbf{y}_T)$ is a **sequence**, and $\mathbf{y}_t$ can be a vector
 - The **length** T may **vary** between datapoints
- This is *not straight forward with FFNNs*
 - Require fixed max length T (need padding)
 - Many unused weights if sequences are short
 - No weight sharing across time steps
 - Issues with temporal dependencies
 - FFNN treats all positions independently
 - Hard to enforce only casual dependencies
 - Cannot generalise naturally to longer sequences

Why not?

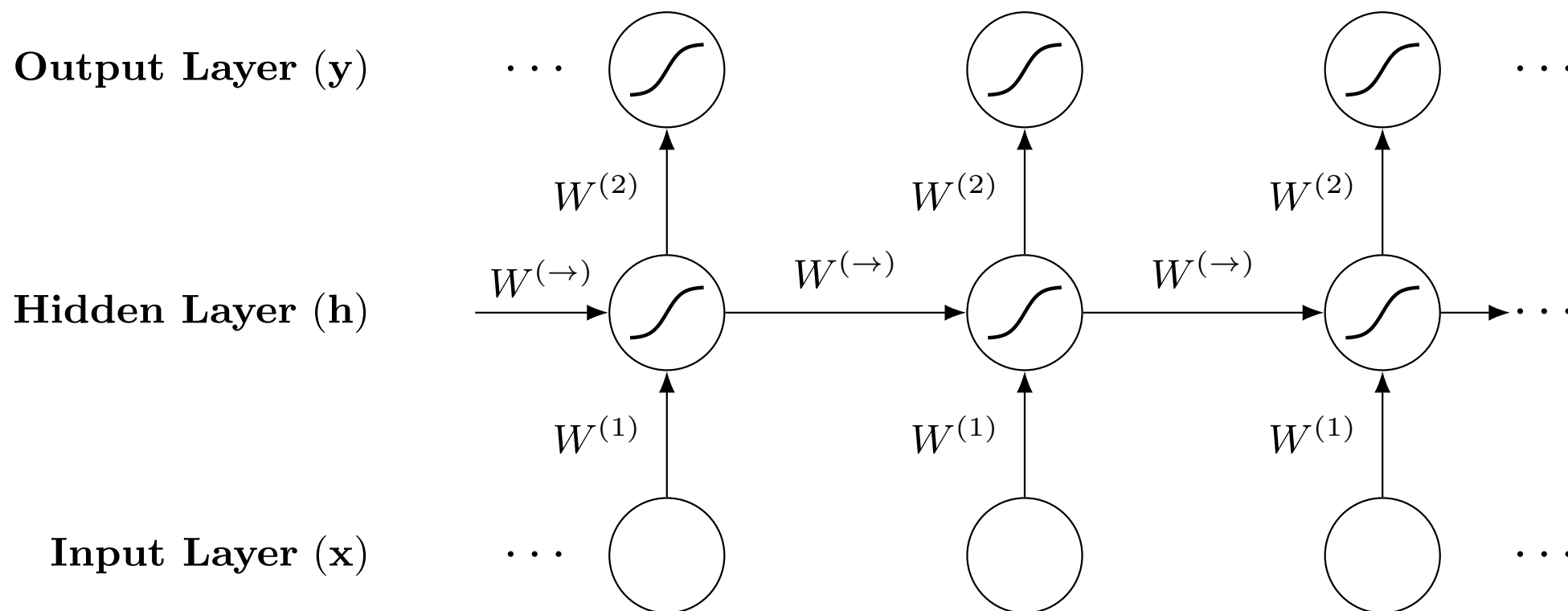


What about CNNs?



- Local receptive field → hard to capture **long-range dependencies**
- **Causality** is not automatic (need masking)

Recurrent neural networks (RNNs)



Input: $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_T)$

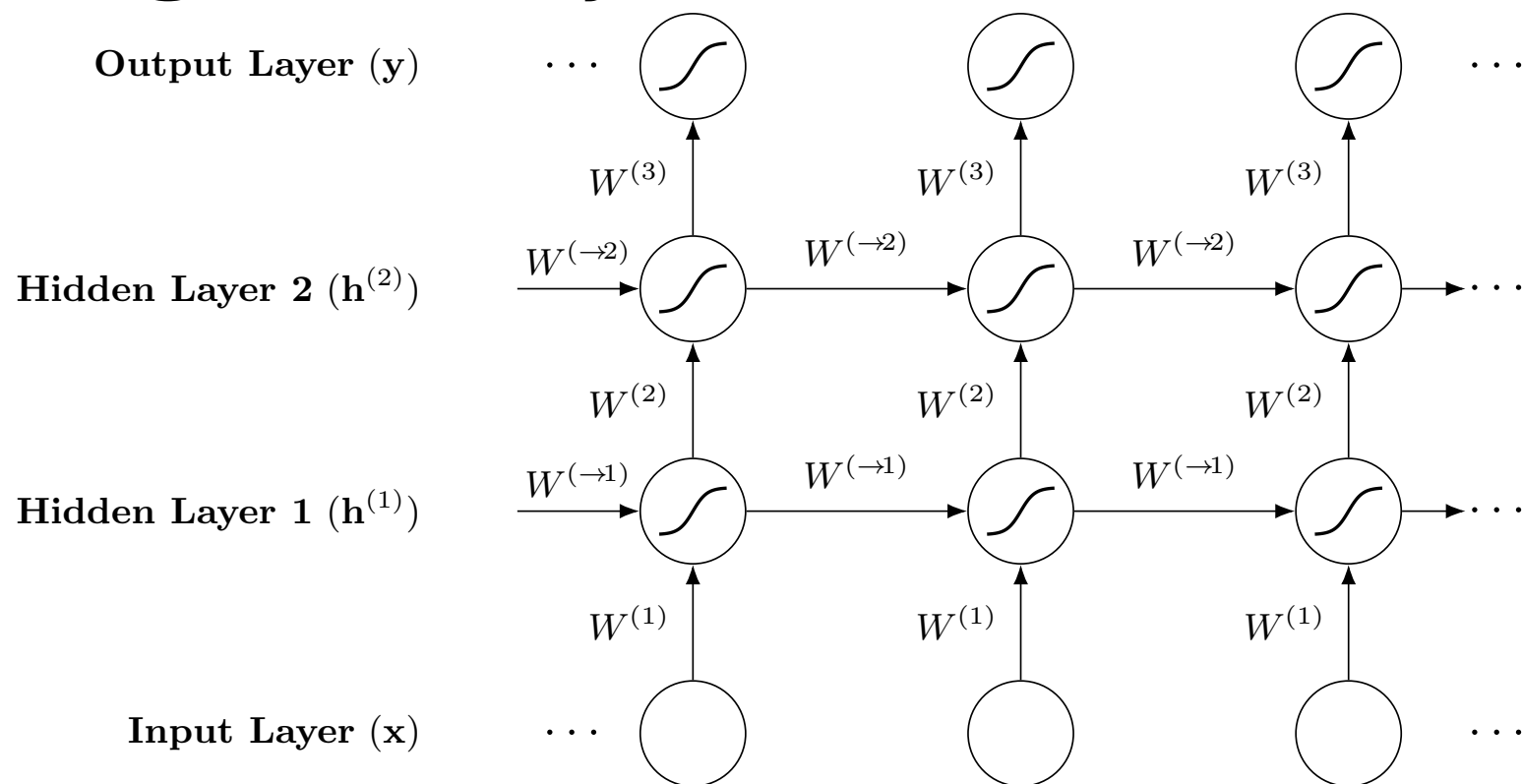
Output: $\hat{\mathbf{y}} = (\hat{\mathbf{y}}_1, \dots, \hat{\mathbf{y}}_T)$

Activations fun: σ_1 and σ_2

$$\mathbf{h}_t = \sigma_1(W^{(1)}\mathbf{x}_t + W^{(\rightarrow)}\mathbf{h}_{t-1} + \mathbf{b}^{(1)} + \mathbf{b}^{(\rightarrow)})$$

$$\hat{\mathbf{y}}_t = \sigma_2(W^{(2)}\mathbf{h}_t + \mathbf{b}^{(2)})$$

Stacking RNN layers



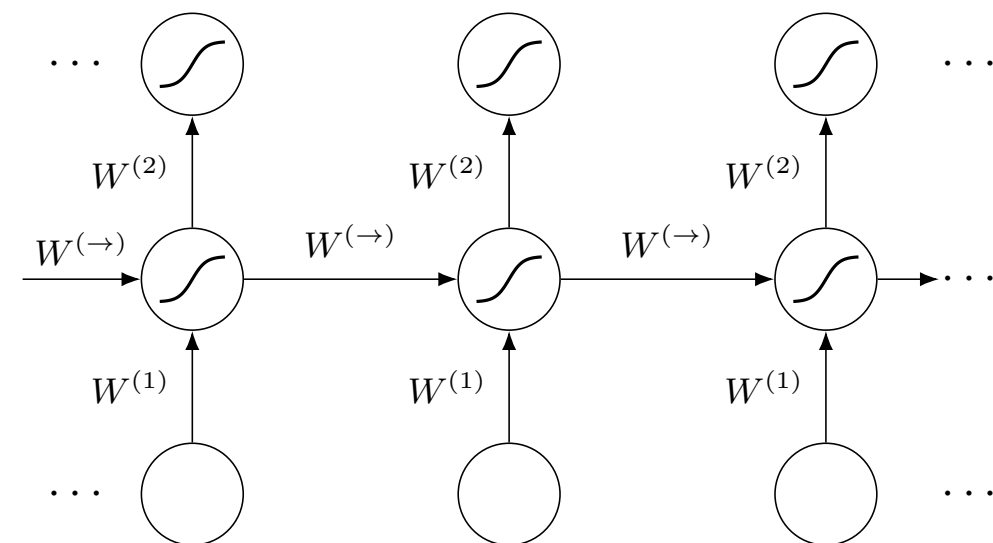
$$\mathbf{h}_t^{(1)} = \sigma_1 \left(W^{(1)} \mathbf{x}_t + W^{(\rightarrow 1)} \mathbf{h}_{t-1}^{(1)} + b^{(1)} + b^{(\rightarrow 1)} \right)$$

$$\mathbf{h}_t^{(2)} = \sigma_2 \left(W^{(2)} \mathbf{h}_t^{(1)} + W^{(\rightarrow 2)} \mathbf{h}_{t-1}^{(2)} + b^{(2)} + b^{(\rightarrow 2)} \right)$$

$$\hat{\mathbf{y}}_t = \sigma_3 \left(W^{(3)} \mathbf{h}_t^{(2)} + b^{(3)} \right)$$

Properties of RNNs

- Handle **variable-length** sequences naturally
- **Share weights** across time steps $\rightarrow$ parameter efficient
- Capture **long-range temporal dependencies** between elements
- Can be constrained to **respect causality**: $\mathbf{y}_t$ depends only on $\mathbf{x}_{\leq t}$
- **Generalise to sequences longer** than those seen during training



- You can think of $\mathbf{h}$ as a memory that accumulate information about previous step
- Note that the network is very deep in the sequence/signal

Loss function:

Maximum likelihood estimation

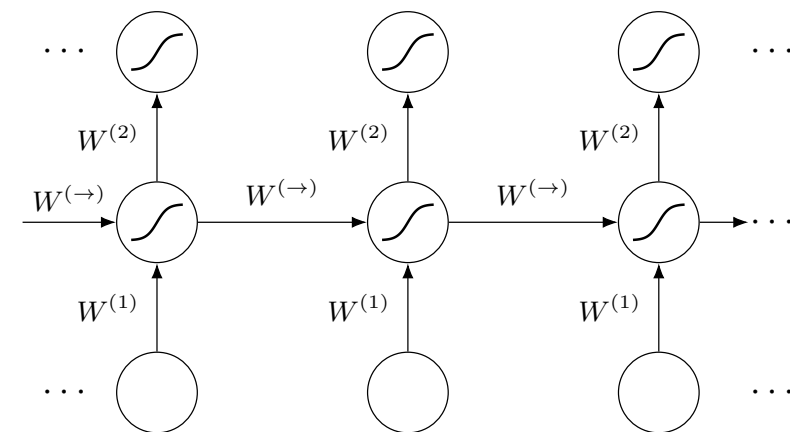
Find **parameters** that maximises the log-probability of the data $\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_i$

$$\hat{\phi} = \arg \max_{\phi} \sum_i \log p(\mathbf{y}^{(i)} | f_{\phi}(\mathbf{x}^{(i)}))$$

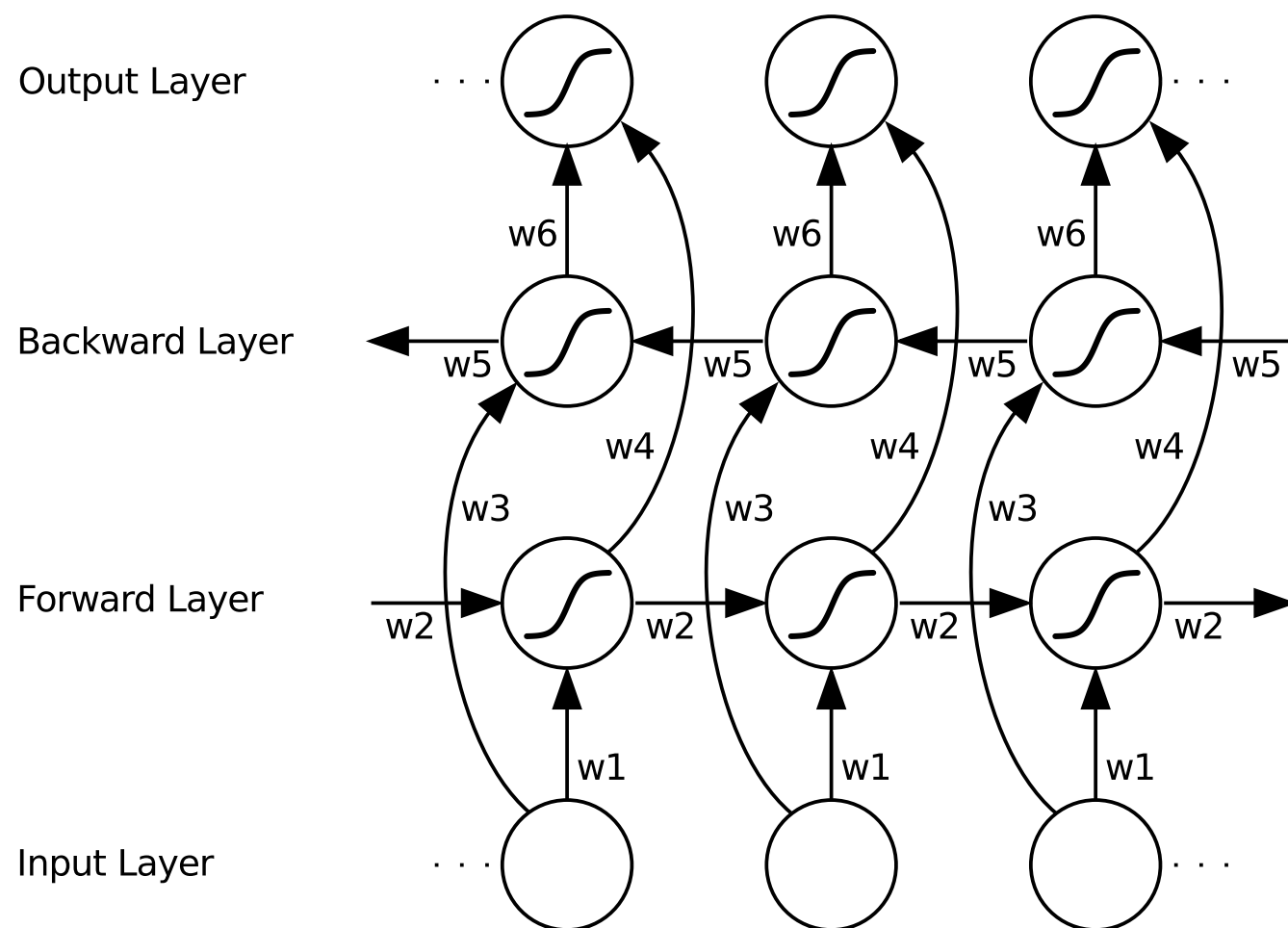
where $\hat{\mathbf{y}}^{(i)} = f_{\phi}(\mathbf{x}^{(i)})$ is the output of an RNN.

For an **input-out pair** $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_T)$ and $\mathbf{y} = (\mathbf{y}_1, \dots, \mathbf{y}_T)$, we usually **assume**

$$\begin{aligned} p(\mathbf{y} | f_{\phi}(\mathbf{x})) &= \prod_{t=1}^T p(\mathbf{y}_t | f_{\phi}(\mathbf{x})_t) \\ &= \prod_{t=1}^T p(\mathbf{y}_t | f_{\phi}(\mathbf{x}_{\leq t})_t) \end{aligned}$$



Bidirectional recurrent neural networks



- An alternative **stacking** of RNNs layers
- Can **integrate** information from both **past** and **future**
- Does **not** respect **causality**

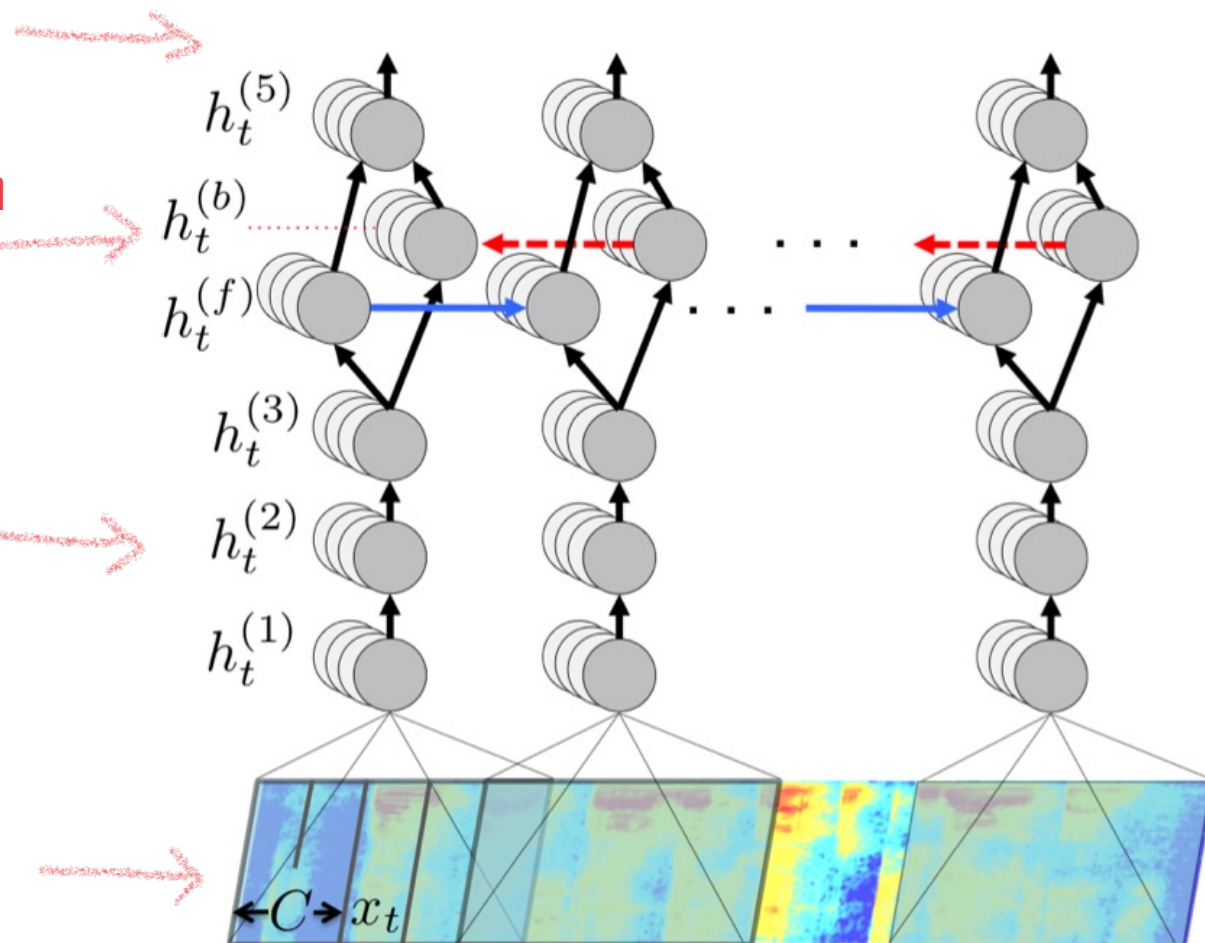
Bidirectional recurrent neural networks: DeepSpeech

English text
transcription

Bidirectional
RNN

CNN

Spectrogram



**DeepSpeech: Scaling up end-to-end
speech recognition**

Awni Hannun*, Carl Case, Jared Casper, Bryan Catanzaro, Greg Diamos, Erich Elsen,
Ryan Prenger, Sanjeev Satheesh, Shubho Sengupta, Adam Coates, Andrew Y. Ng

Baidu Research – Silicon Valley AI Lab

Varying size input and output

- We want to do supervised learning on data pairs $(\mathbf{x}, \mathbf{y})$, where
 - The **input** $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_T)$ is a **sequence**
 - The **output** $\mathbf{y} = (\mathbf{y}_1, \dots, \mathbf{y}_S)$ is a **sequence**
- The **length** T and S may **vary** between datapoints
- **The length of the two sequence may differ, $T \neq S$**

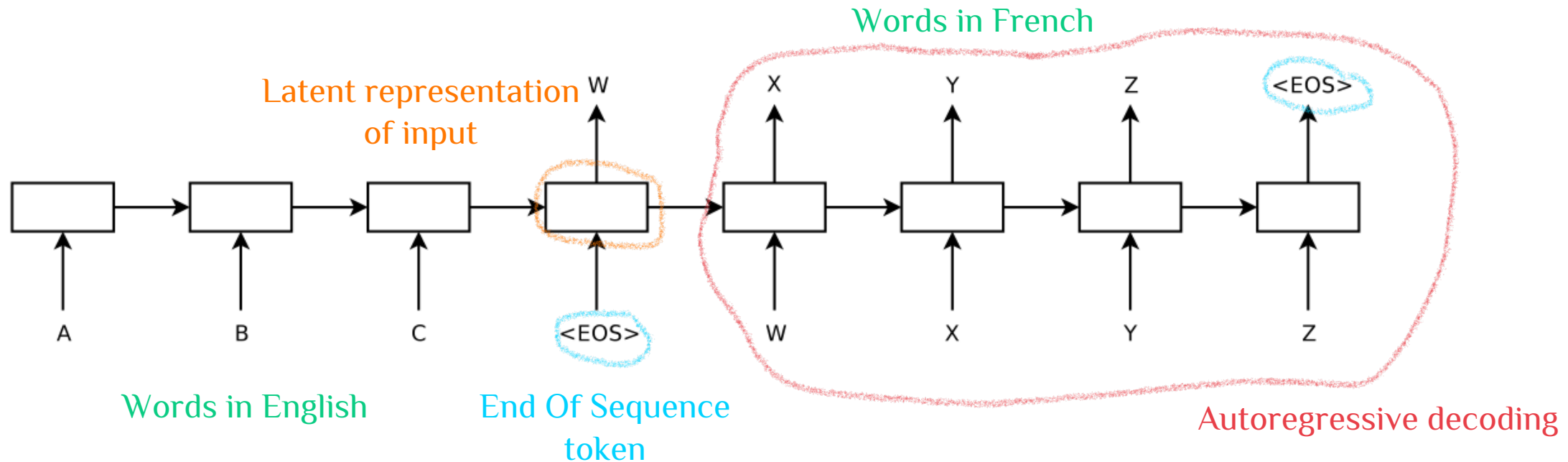
Encoder-decoder: machine translation

Sequence to Sequence Learning with Neural Networks

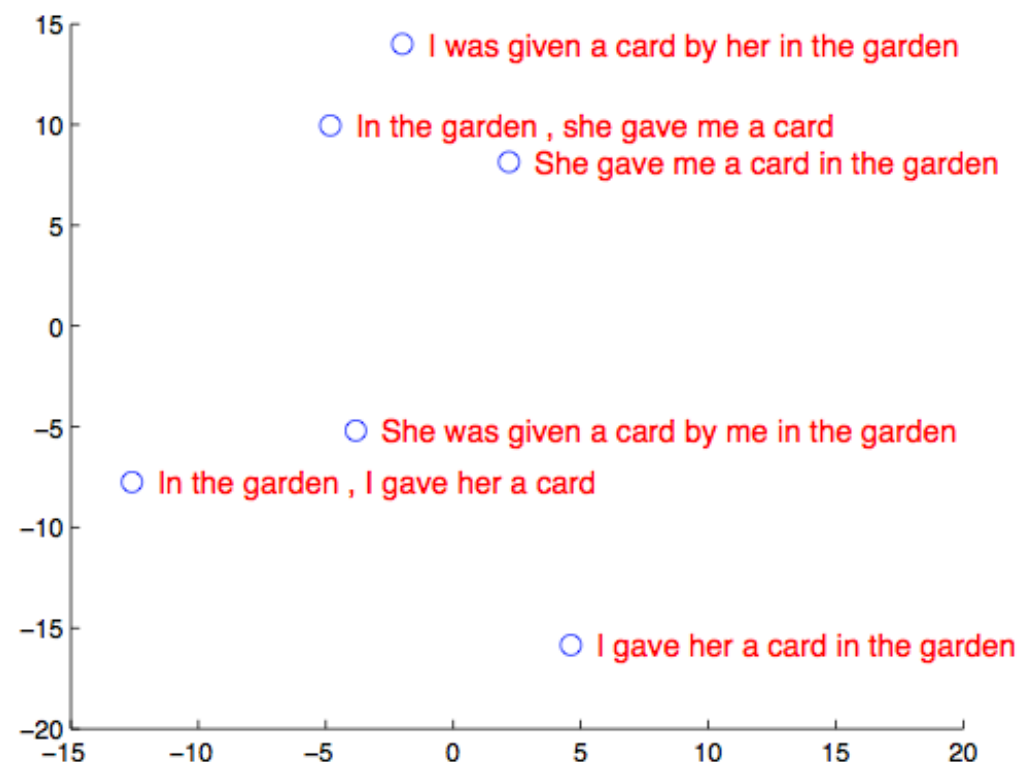
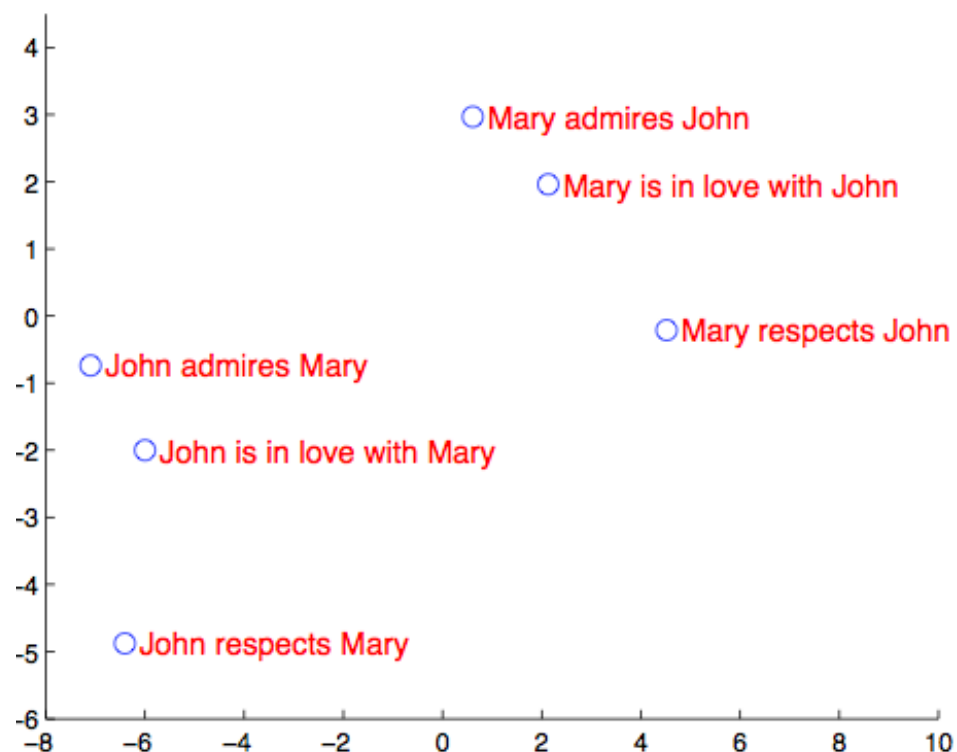
Ilya Sutskever
Google
ilyasu@google.com

Oriol Vinyals
Google
vinyals@google.com

Quoc V. Le
Google
qvl@google.com



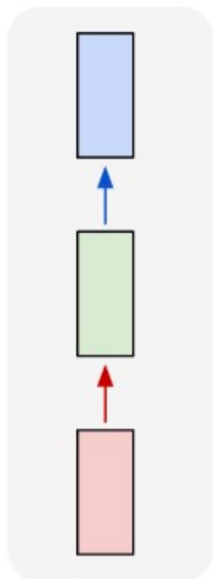
Machine translation: visualizing the latent representation



What is the difference between the two last?

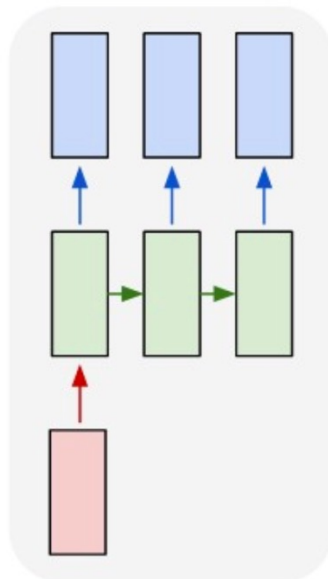
Overview recurrent architectures

one to one



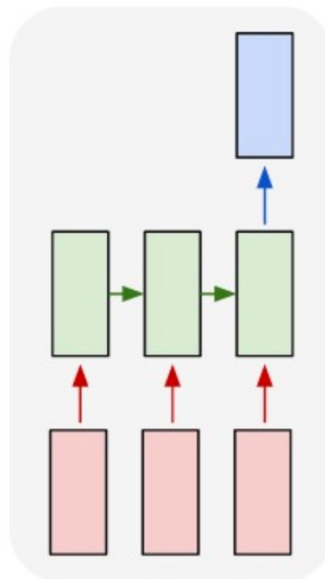
- Vanilla mode, no RNN.
- E.g. image classification

one to many



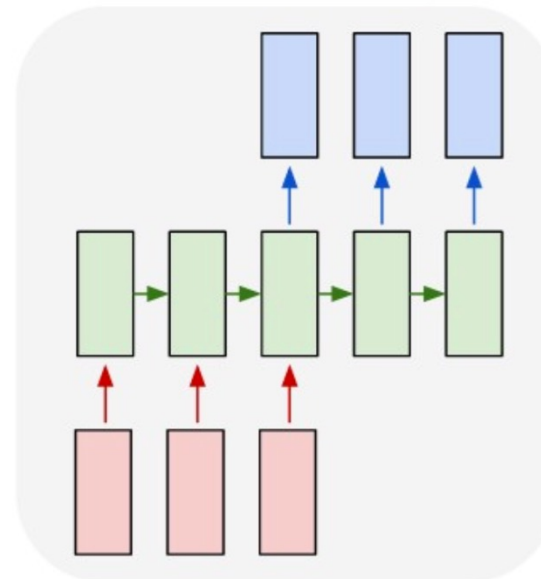
- Sequence output
- E.g. image captioning

many to one



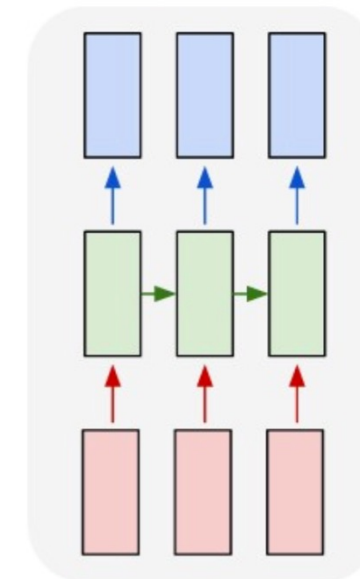
- Sequence input
- E.g. sentiment analysis

many to many



- Sequence input and output (*encoder-decoder, sequence-to-sequence*)
- E.g. translation, question answering

many to many



- Synced sequence input and output
- E.g. label each video frame

Vanishing and exploding gradients

- The recurrence relation in RNNs resembles

$$\mathbf{h}_t = W\mathbf{h}_{t-1}$$

(ignoring the input, bias and activation)

- This means that

$$\mathbf{h}_t = W^t \mathbf{h}_0$$

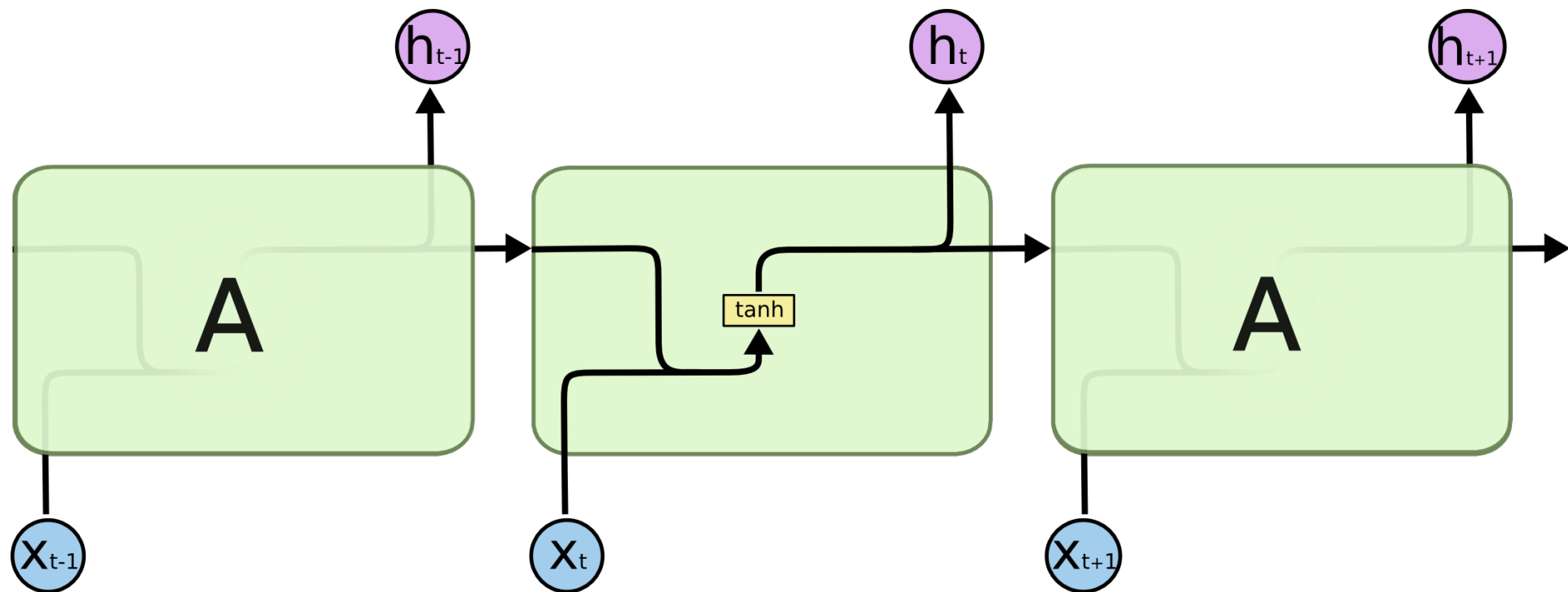
- If we do an eigenvalue decomposition $W = Q\Lambda Q^T$, we have

$$\mathbf{h}_t = Q\Lambda^t Q^T \mathbf{h}_0$$

- Eigenvalues less one decay to zero
 - Eigenvalues greater than one explode
- Causes same behaviour of gradient

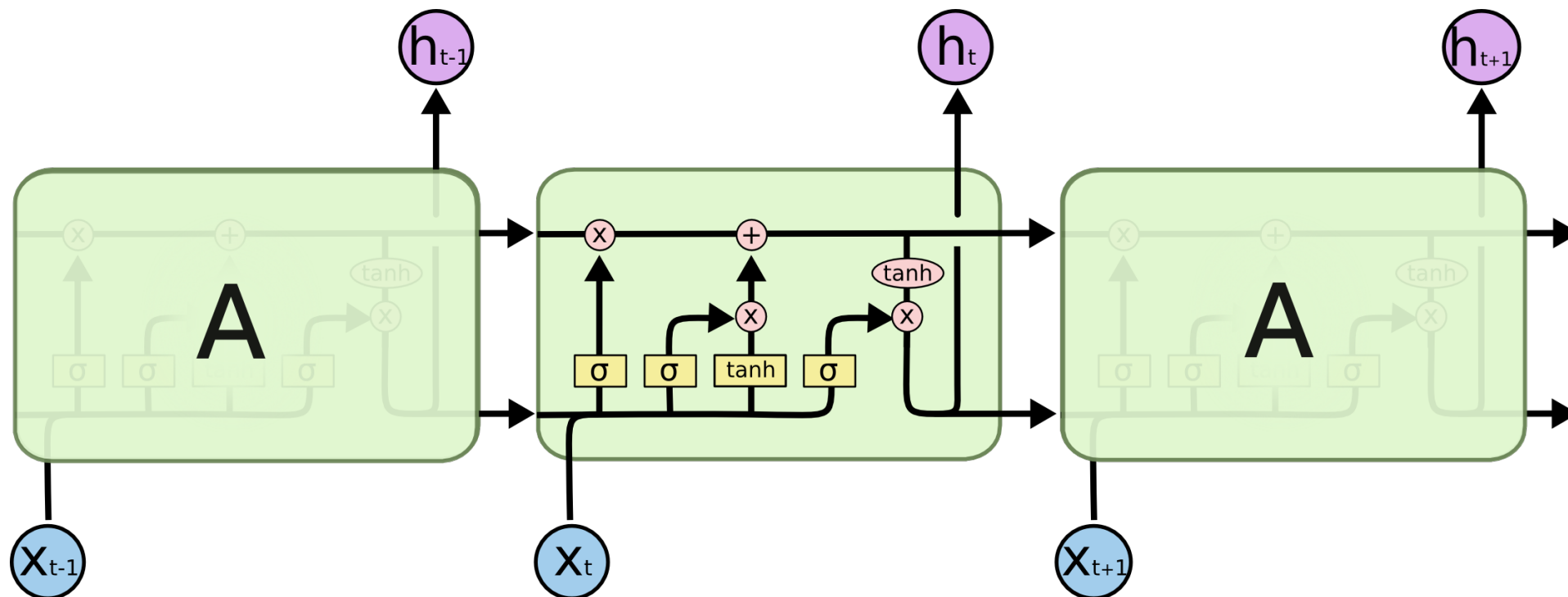
Long short-term memory (LSTM) cells

Regular RNN



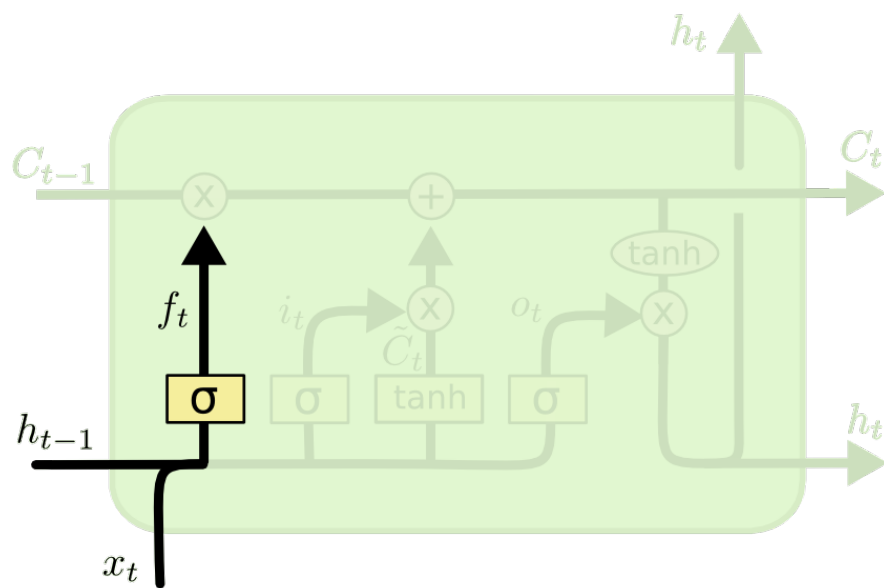
Long short-term memory (LSTM) cells

LSTM cell



Long short-term memory (LSTM) cells

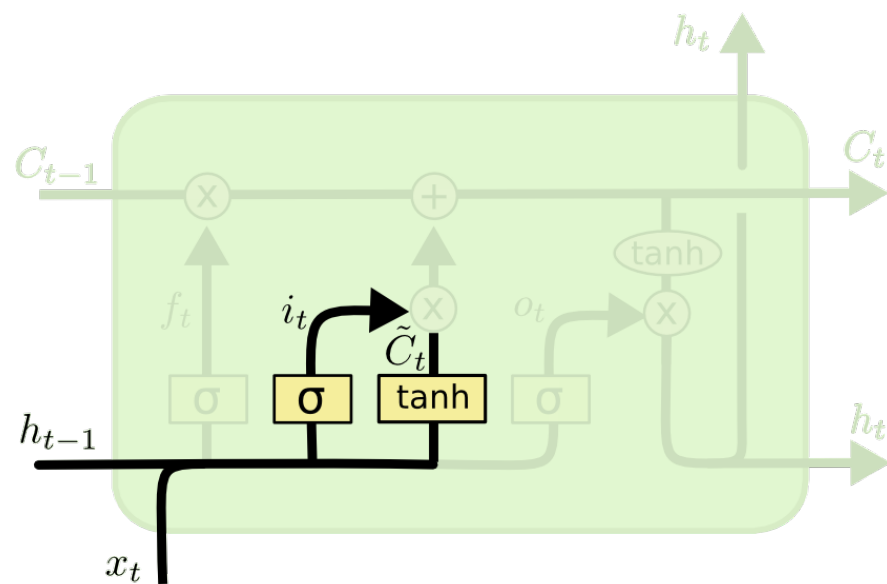
Forget gate f_t



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

Long short-term memory (LSTM) cells

Input gate i_t and new candidate cell state $\tilde{C}_t$

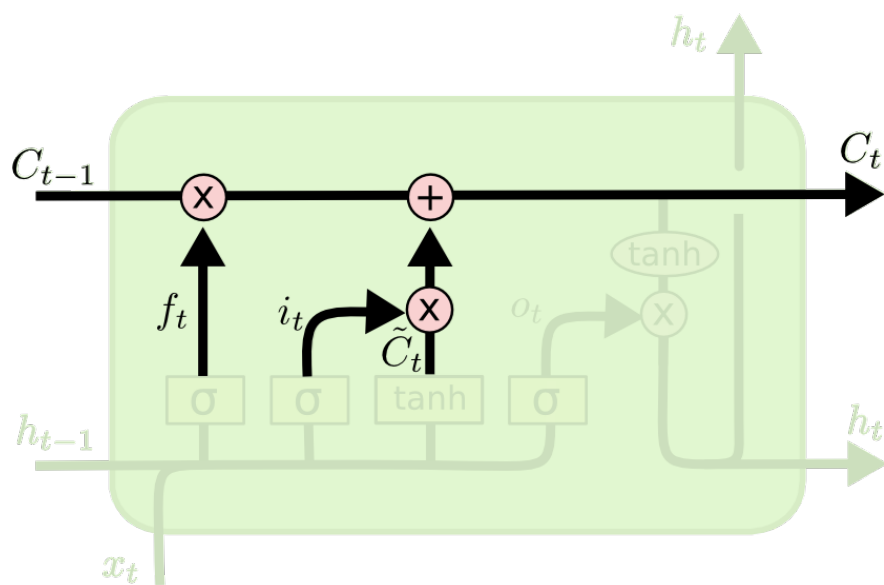


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Long short-term memory (LSTM) cells

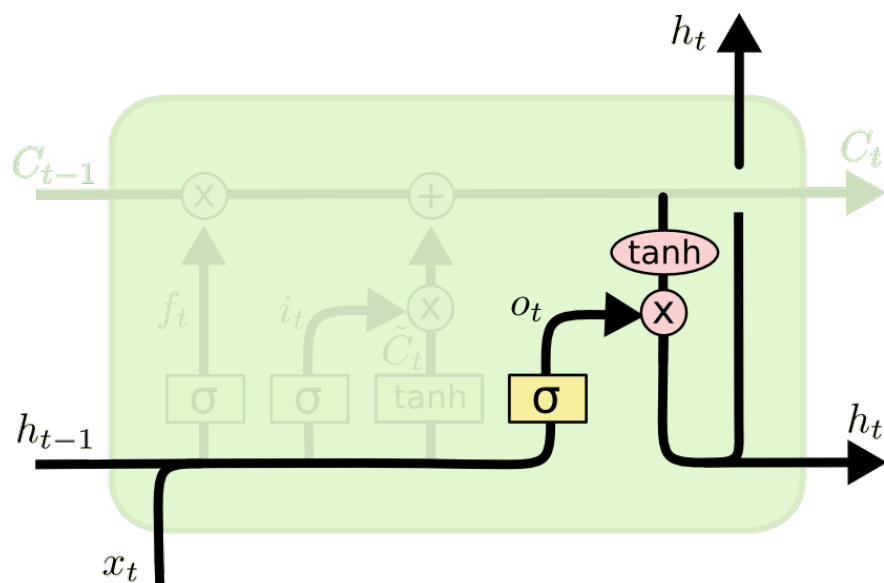
New cell state C_t



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Long short-term memory (LSTM) cells

Output gate o_t and new output h_t



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

Something about LSTM

- **Preserve gradient flow** via cell state → mitigate vanishing gradients
- **Capture long-term dependencies**, effective for sequential tasks
- **Predecessor to Transformers**
 - less parallel but still **useful in low-resource settings**

Today's exercises!

- Three Python notebooks ([Notebook 5.1](#))
 - Vanilla RNN with Nanograd
 - LSTM with Nanograd & **PyTorch**!
- No text-book **problems** this week

Thank you!